

Câu lệnh rẽ nhánh (If/Else)

Nội dung

1. Toán tử so sánh
2. Câu lệnh if
3. Câu lệnh if-else
4. Chuỗi if-elif-else (nhiều nhánh)
5. Toán tử logic
6. Câu lệnh if lồng nhau
7. Các mẫu (pattern) thường gặp
 - Tìm số lớn nhất (max) của hai số
 - Tìm số lớn nhất của ba số
 - Sắp xếp 2 số (in theo thứ tự tăng dần)
 - Kiểm tra năm nhuận
 - Kiểm tra số chính phương
8. Mẹo và lỗi thường gặp
 - Lỗi 1: Nhầm = (gán) với == (so sánh)
 - Lỗi 2: Thiếu ngoặc nhọn trong C++
 - Lỗi 3: Thiếu thụt lề (indent) trong Python
 - Lỗi 4: Sai logic khi kết hợp điều kiện
 - Lỗi 5: So sánh số thực
 - Mẹo chung

1. Toán tử so sánh

Trước khi học `if`, ta cần biết cách so sánh các giá trị. Kết quả của phép so sánh là đúng (`true`) hoặc sai (`false`).

TOÁN TỬ	Ý NGHĨA	VÍ DỤ (TRUE)
<code>==</code>	Bằng nhau	<code>5 == 5</code>
<code>!=</code>	Khác nhau	<code>5 != 3</code>
<code><</code>	Nhỏ hơn	<code>3 < 5</code>
<code>></code>	Lớn hơn	<code>5 > 3</code>
<code><=</code>	Nhỏ hơn hoặc bằng	<code>5 <= 5</code>
<code>>=</code>	Lớn hơn hoặc bằng	<code>5 >= 3</code>

Lưu ý quan trọng: `==` là so sánh, `=` là gán. Đây là lỗi rất phổ biến!

2. Câu lệnh if

Cú pháp cơ bản:

C++

```
if (điều_kiện) {  
    // Thực hiện nếu điều kiện đúng  
}
```

Python

```
if điều_kiện:  
    # Thực hiện nếu điều kiện đúng (thụt vào 4 dấu cách)
```

Ví dụ: Kiểm tra số dương:

C++

```
#include <iostream>  
using namespace std;  
int main() {  
    int n;  
    cin >> n;  
    if (n > 0) {  
        cout << "So duong" << endl;  
    }  
    return 0;  
}
```

Python

```
n = int(input())  
if n > 0:  
    print("So duong")
```

3. Câu lệnh if-else

Khi muốn xử lý cả hai trường hợp — điều kiện đúng và điều kiện sai:

C++

```
if (điều_kiện) {  
    // Thực hiện nếu ĐÚNG  
} else {  
    // Thực hiện nếu SAI  
}
```

Python

```
if điều_kiện:
    # Thực hiện nếu ĐÚNG
else:
    # Thực hiện nếu SAI
```

Ví dụ: Kiểm tra chẵn hay lẻ:

C++

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    if (n % 2 == 0) {
        cout << "Chan" << endl;
    } else {
        cout << "Le" << endl;
    }
    return 0;
}
```

Python

```
n = int(input())
if n % 2 == 0:
    print("Chan")
else:
    print("Le")
```

Giải thích: $n \% 2$ là phần dư khi chia n cho 2. Nếu dư 0 thì n chẵn, ngược lại n lẻ.

4. Chuỗi if-elif-else (nhiều nhánh)

Khi có nhiều trường hợp cần xét:

C++

```
if (điều_kiện_1) {
    // Trường hợp 1
} else if (điều_kiện_2) {
    // Trường hợp 2
} else if (điều_kiện_3) {
    // Trường hợp 3
} else {
```

```
// Trường hợp còn lại  
}
```

Python

```
if điều_kiện_1:  
    # Trường hợp 1  
elif điều_kiện_2:  
    # Trường hợp 2  
elif điều_kiện_3:  
    # Trường hợp 3  
else:  
    # Trường hợp còn lại
```

Ví dụ: Xếp loại điểm:

C++

```
#include <iostream>  
using namespace std;  
int main() {  
    int diem;  
    cin >> diem;  
    if (diem >= 9) {  
        cout << "Xuat sac" << endl;  
    } else if (diem >= 7) {  
        cout << "Kha" << endl;  
    } else if (diem >= 5) {  
        cout << "Trung binh" << endl;  
    } else {  
        cout << "Yeu" << endl;  
    }  
    return 0;  
}
```

Python

```
diem = int(input())  
if diem >= 9:  
    print("Xuat sac")  
elif diem >= 7:  
    print("Kha")  
elif diem >= 5:  
    print("Trung binh")  
else:  
    print("Yeu")
```

Lưu ý: Các điều kiện được kiểm tra từ trên xuống. Khi một điều kiện đúng, các `elif` / `else if` bên dưới sẽ bị bỏ qua. Vì vậy, ta không cần viết `diem >= 7 && diem < 9` — chỉ cần `diem >= 7` vì nếu `diem >= 9` thì đã rẽ vào nhánh trên rồi.

5. Toán tử logic

Khi cần kết hợp nhiều điều kiện:

TOÁN TỬ	C++	PYTHON	Ý NGHĨA
VÀ (AND)	<code>&&</code>	<code>and</code>	Đúng khi cả hai đều đúng
HOẶC (OR)	<code> </code>	<code>or</code>	Đúng khi ít nhất một đúng
PHỦ ĐỊNH (NOT)	<code>!</code>	<code>not</code>	Đảo ngược: đúng thành sai, sai thành đúng

Ví dụ: Kiểm tra số trong đoạn [1, 100]:

C++

```
int n;
cin >> n;
if (n >= 1 && n <= 100) {
    cout << "Trong doan [1, 100]" << endl;
} else {
    cout << "Ngoai doan" << endl;
}
```

Python

```
n = int(input())
if n >= 1 and n <= 100:
    print("Trong doan [1, 100]")
else:
    print("Ngoai doan")
```

Bảng chân trị (truth table):

A	B	A AND B	A OR B	NOT A
true	true	true	true	false
true	false	false	true	false
false	true	false	true	true
false	false	false	false	true

6. Câu lệnh if lồng nhau

Ta có thể đặt `if` bên trong `if` khác:

C++

```
#include <iostream>
using namespace std;
int main() {
    int a, b, c;
    cin >> a >> b >> c;
    if (a + b > c && b + c > a && a + c > b) {
        // Là tam giác - xét loại
        if (a == b && b == c) {
            cout << "Tam giác đều" << endl;
        } else if (a == b || b == c || a == c) {
            cout << "Tam giác cân" << endl;
        } else {
            cout << "Tam giác thường" << endl;
        }
    } else {
        cout << "Không là tam giác" << endl;
    }
    return 0;
}
```

Python

```
a, b, c = map(int, input().split())
if a + b > c and b + c > a and a + c > b:
    # Là tam giác - xét loại
    if a == b == c:
        print("Tam giác đều")
    elif a == b or b == c or a == c:
        print("Tam giác cân")
    else:
        print("Tam giác thường")
else:
    print("Không là tam giác")
```

7. Các mẫu (pattern) thường gặp

Tìm số lớn nhất (max) của hai số

C++

```
int a, b;
cin >> a >> b;
int max_val;
if (a > b) {
    max_val = a;
} else {
    max_val = b;
}
cout << max_val << endl;
```

```
// Hoặc dùng hàm có sẵn:  
cout << max(a, b) << endl;
```

Python

```
a, b = map(int, input().split())  
if a > b:  
    max_val = a  
else:  
    max_val = b  
print(max_val)  
# Hoặc dùng hàm có sẵn:  
print(max(a, b))
```

Tìm số lớn nhất của ba số

C++

```
int a, b, c;  
cin >> a >> b >> c;  
cout << max({a, b, c}) << endl;  
// Hoặc tự viết:  
int m = a;  
if (b > m) m = b;  
if (c > m) m = c;  
cout << m << endl;
```

Python

```
a, b, c = map(int, input().split())  
print(max(a, b, c))  
# Hoặc tự viết:  
m = a  
if b > m:  
    m = b  
if c > m:  
    m = c  
print(m)
```

Sắp xếp 2 số (in theo thứ tự tăng dần)

C++

```
int a, b;  
cin >> a >> b;  
if (a > b) {  
    // Hoán đổi (swap)  
    int temp = a;  
    a = b;  
    b = temp;  
}
```

```
// Hoặc: swap(a, b);
}
cout << a << " " << b << endl;
```

Python

```
a, b = map(int, input().split())
if a > b:
    a, b = b, a # Python swap rất gọn!
print(a, b)
```

Kiểm tra năm nhuận

Một năm là năm nhuận nếu:

- Chia hết cho 4 VÀ không chia hết cho 100, HOẶC
- Chia hết cho 400.

C++

```
int year;
cin >> year;
if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0) {
    cout << "Leap year" << endl;
} else {
    cout << "Not leap year" << endl;
}
```

Python

```
year = int(input())
if (year % 4 == 0 and year % 100 != 0) or year % 400 == 0:
    print("Leap year")
else:
    print("Not leap year")
```

Kiểm tra số chính phương

Một số n là số chính phương nếu tồn tại số nguyên k sao cho $k^2 = n$.

C++

```
#include <iostream>
#include <cmath>
using namespace std;
int main() {
    int n;
    cin >> n;
    int k = (int)sqrt((double)n);
    // Kiểm tra cả k và k+1 để tránh sai số làm tròn của sqrt
```

```

    if (k * k == n || (k + 1) * (k + 1) == n) {
        cout << "YES" << endl;
    } else {
        cout << "NO" << endl;
    }
    return 0;
}

```

Python

```

import math
n = int(input())
k = int(math.sqrt(n))
# Kiểm tra cả k và k+1 để tránh sai số làm tròn
if k * k == n or (k + 1) * (k + 1) == n:
    print("YES")
else:
    print("NO")

```

8. Mẹo và lỗi thường gặp

Lỗi 1: Nhầm = (gán) với == (so sánh)

C++

```

// SAI - đây là phép gán, không phải so sánh!
if (x = 5) { // Luôn "đúng" vì gán x = 5, giá trị 5 != 0
    cout << "x bang 5" << endl;
}
// ĐÚNG
if (x == 5) {
    cout << "x bang 5" << endl;
}

```

Trong Python, `if x = 5:` sẽ gây lỗi cú pháp (`SyntaxError`), nên dễ phát hiện hơn.

Lỗi 2: Thiếu ngoặc nhọn trong C++

C++

```

// SAI - chỉ dòng đầu tiên thuộc if, dòng thứ hai LUÔN chạy!
if (x > 0)
    cout << "Duong" << endl;
    cout << "Chac chan duong" << endl; // Dòng này luôn chạy!
// ĐÚNG - luôn dùng ngoặc nhọn
if (x > 0) {
    cout << "Duong" << endl;
    cout << "Chac chan duong" << endl;
}

```

```
}
```

Mẹo: Luôn dùng {} trong C++ dù chỉ có 1 dòng lệnh — tránh bug khó tìm.

Lỗi 3: Thiếu thụt lề (indent) trong Python

Python

```
# SAI - Python dựa vào thụt lề, thiếu sẽ lỗi!  
if x > 0:  
print("Duong") # IndentationError!  
# ĐÚNG  
if x > 0:  
    print("Duong") # Thụt vào 4 dấu cách
```

Lỗi 4: Sai logic khi kết hợp điều kiện

C++

```
// SAI - muốn kiểm tra x trong [1, 10] nhưng viết sai  
if (x >= 1 || x <= 10) // Điều kiện này LUÔN đúng với mọi x!  
// ĐÚNG  
if (x >= 1 && x <= 10) // Dùng AND, không phải OR
```

Lỗi 5: So sánh số thực

Số thực có sai số làm tròn, nên so sánh == có thể sai:

C++

```
double a = 0.1 + 0.2;  
// SAI - a có thể là 0.30000000000000004  
if (a == 0.3) { ... }  
// ĐÚNG - so sánh với sai số nhỏ (epsilon)  
if (abs(a - 0.3) < 1e-9) { ... }
```

Mẹo chung

- Luôn xét các trường hợp biên: số 0, số âm, giá trị min/max.
- Khi có nhiều nhánh, vẽ ra giấy hoặc liệt kê tất cả trường hợp trước khi code.
- Kiểm tra kỹ thứ tự các điều kiện trong if-elif — điều kiện chặt hơn nên đứng trước.
- Test chương trình với nhiều input khác nhau, đặc biệt các trường hợp giáp ranh.